DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSA0607 – DESIGN AND ANALYSIS OF ALGORITHMS

CO1 – ASSESSMENT TOOL 1

IT APPLICATION-BASED PROBLEM SOLVING

SCALABILITY AUDIT OF AN E-COMMERCE PLATFORM

Platform: ShopSphere

Student Name: K. Hasitha Sai Sree

Registration Number: 192371065

1. Objective

The objective is to audit the algorithmic scalability of ShopSphere as its product catalogue grows from 100,000 to 1,000,000 products. The analysis identifies dominant operations, derives exact counts where possible, solves recurrences, applies asymptotic notation, and estimates whether each module can finish within the required 10-minute nightly processing window.

2. Problem Assumptions

- Current catalogue size: $n = 100{,}000$ products.
- Expected catalogue size: $n = 1{,}000{,}000$ products.
- Server rate given in the question: 10 basic operations per second.
- Nightly processing window: 10 minutes = 600 seconds = 6,000 basic operations at the stated rate.
- M1 compares every product with every later product.
- M2 doubles i each iteration and audits $j = 0$ through $i - 1$.
- S2 uses binary search on a sorted index and follows $T(n) = T(n/2) + 1$, $T(1) = 1$.
- M4 current: $T(n) = 4T(n/2) + n$; proposed: $T(n) = 2T(n/2) + n$.
- M5: $T(n) = 2T(n/2) + n \log n$; report generator: $f(n) = 3n^2 + 10n \log n + 50$.

3. (a) Understand the Problem

Module	Input size	Dominant basic operation	Expected growth
M1 Duplicate detector	n product records	Compare title and seller for each pair (i, j)	Quadratic: $\Theta(n^2)$
M2 API rate limiter	n request-window entries	Audit(Request[j]) in geometrically growing windows	Linear: $\Theta(n)$
M3 S1 sequential search	n catalogue entries	Compare target with each product	Best $\Theta(1)$, average/worst $\Theta(n)$
M3 S2 binary search	n sorted index entries	Compare target with middle entry	$\Theta(\log n)$
M4 recommendation	n customer–product records	Recursive subproblem work plus linear merge	Current $\Theta(n^2)$; redesign $\Theta(n \log n)$
M5 analytics	n sales records	Recursive split plus n log n sorting work	$\Theta(n(\log n)^2)$

The report generator is also audited separately in part (f). Its dominant term is $3n^2$, so its growth is $\Theta(n^2)$.

4. (b) Analyse the Iterative Modules

4.1 M1 — Duplicate-Listing Detector

The outer loop chooses $i = 0, 1, \ldots, n - 2$. For each i, the inner loop compares product i with all later products.

Comparisons = (n − 1) + (n − 2) + ... + 2 + 1

= 1 + 2 + ... + (n − 1)

= n(n − 1) / 2

For n = 100,000: 100,000 × 99,999 / 2 = 4,999,950,000 comparisons.

For n = 1,000,000: 1,000,000 × 999,999 / 2 = 499,999,500,000 comparisons.

Time complexity: $\Theta(n^2)$, because the exact comparison count is a quadratic polynomial. Auxiliary space is O(1) if duplicate marks are written directly to existing records; if the implementation stores a separate list of duplicate IDs, that output storage can require O(n) space.

4.2 M2 — API Rate Limiter

The outer-loop values of i are 1, 2, 4, 8, … while i < n. Each iteration performs exactly i Audit operations.

Total = 1 + 2 + 4 + ... + 2^(m−1)

= (2^m − 1) / (2 − 1) = 2^m − 1

The number of iterations is m = ceil($\log_2$ n), so the exact count is 2^ceil($\log_2$ n) − 1. Because 2^ceil($\log_2$ n) is at least n and less than 2n for n > 1, the count is $\Theta(n)$, not $\Theta(n \log n)$. The team member's claim is incorrect: the inner-loop work is not n on every outer iteration; it is 1, 2, 4, 8, ….

For n = 1,024, m = $\log_2(1{,}024)$ = 10; total = $2^{10} - 1$ = 1,023 Audit operations.

For n = 1,000,000, ceil($\log_2$ n) = 20; total = $2^{20} - 1$ = 1,048,575 Audit operations.

5. (c) Solve S2 by Backward Substitution

Given T(n) = T(n/2) + 1 and T(1) = 1, assume n is a power of two for the exact closed form.

T(n) = T(n/2) + 1

= [T(n/4) + 1] + 1 = T(n/4) + 2

= T(n/8) + 3

= ... = T(n/2^k) + k

Stop when n/2^k = 1. Therefore n = 2^k and k = $\log_2$ n.

T(n) = T(1) + $\log_2$ n = 1 + $\log_2$ n

Thus $T(n) = \Theta(\log n)$. For n = 1,000,000, $\log_2$ n ≈ 19.93, so the comparison estimate is about 20.

Space: iterative binary search uses $\Theta(1)$ auxiliary space because it reuses a fixed number of variables. Recursive binary search uses $\Theta(\log n)$ stack space because the recursion depth is logarithmic.

6. (d) Apply the Master Theorem

6.1 M4 — Current recommendation engine

T(n) = 4T(n/2) + n

a = 4, b = 2, f(n) = n.

n^(log_b a) = n^($\log_2$ 4) = n^2

Since f(n) = n = O(n^(2−1)), f(n) is polynomially smaller than n^2. Master Theorem Case 1 applies.

$T(n) = \Theta(n^2)$

When input size doubles, a quadratic cost grows by approximately $2^2 = 4$ times.

6.2 M4 — Proposed redesign

$T(n) = 2T(n/2) + n$

a = 2, b = 2, f(n) = n.

$n^{(\log_b a)} = n^{(\log_2 2)} = n$

Here $f(n) = \Theta(n^{(\log_b a)})$, so Master Theorem Case 2 applies.

$T(n) = \Theta(n \log n)$

When n doubles, the dominant expression changes by approximately $2 \times \log_2(2n)/\log_2(n)$, which is a little more than 2 and approaches 2 for large n. This is substantially better than the current design's approximately fourfold increase.

6.3 M5 — Sales analytics engine

$T(n) = 2T(n/2) + n \log n$

a = 2, b = 2, and $n^{(\log_b a)} = n$. Here $f(n) = \Theta(n \log n)$, which is not $\Theta(n)$ and does not fit the three basic cases in their simplest forms.

Using the extended Master Theorem (Case 2 with $f(n) = \Theta(n^{\log_b(a)} \log^k n)$, k = 1), $T(n) = \Theta(n \log^{(k+1)} n) = \Theta(n(\log n)^2)$.

7. (e) Best, Average, and Worst Search Cases

7.1 S1 — Sequential search

- Best case: target is the first item; 1 comparison.
- Average successful case: assuming each position is equally likely, $(1 + 2 + \ldots + n)/n = (n + 1)/2$ comparisons.
- Worst case: target is last or absent; n comparisons.

Let p be the probability that the target exists. If it exists, expected comparisons are (n + 1)/2; if it does not exist, all n items are checked. Therefore:

$E[C] = p(n + 1)/2 + (1 - p)n$

For n = 1,000,000 and p = 0.5:

$E[C] = 0.5(1{,}000{,}001)/2 + 0.5(1{,}000{,}000)$

$= 250{,}000.25 + 500{,}000 = 750{,}000.25$ comparisons

7.2 S2 — Binary search

- Best case: target equals the first middle element; 1 comparison.
- Average case: the target is found after a logarithmic number of halvings; $\Theta(\log n)$ comparisons.
- Worst case: the search follows the maximum-height path or reports absence; at most $\text{floor}(\log_2 n) + 1$ comparisons for the usual iterative implementation.

Binary search requires sorted data; maintaining the sorted index is a separate cost not included in the search comparison count.

8. (f) Prove $f(n) = \Theta(n^2)$ and $f(n)$ is not $O(n)$

Given $f(n) = 3n^2 + 10n \log_2 n + 50$, for $n \geq 1$, $\log_2 n \leq n$.

Lower bound: $f(n) = 3n^2 + 10n \log_2 n + 50 \geq 3n^2$

Upper bound: $f(n) \leq 3n^2 + 10n^2 + 50n^2 = 63n^2$

Choose $c_1 = 3$, $c_2 = 63$, and $n_0 = 1$. Then $0 \leq c_1 n^2 \leq f(n) \leq c_2 n^2$ for all $n \geq n_0$. By the formal definition, $f(n) = \Theta(n^2)$.

To show $f(n)$ is not $O(n)$, assume a constant c could satisfy $f(n) \leq cn$ for all sufficiently large n. But $f(n)/n \geq 3n$, which grows without bound as n increases. No fixed c can bound this ratio; therefore $f(n)$ is not $O(n)$.

9. (g) Scalability Decision at n = 1,000,000

At 10 operations/second, a 10-minute window can accommodate only 6,000 basic operations.

Module / model	Operation estimate	Estimated time	10-minute window?
M1: n(n−1)/2	499,999,500,000	about 1,584.40 years	No
M2: 2^ceil(log₂n)−1	1,048,575	about 1.21 days	No
S2: binary search	about 20 comparisons	about 2 seconds	Yes
M4 current: n² proxy	about 1,000,000,000,000	about 3,168.81 years	No
M4 redesign: n log₂n proxy	about 19,931,569	about 23.07 days	No at 10 ops/s
M5: n(log₂n)² proxy	about 397,267,426	about 1.26 years	No

Important estimation note: M1, M2 and S2 use the counts derived from the specified algorithms. For M4 and M5, the question gives recurrences but not base-case costs or the exact mapping between recurrence work and the phrase “basic operation.” Their totals are therefore explicitly labelled asymptotic proxies, not exact counts or measured execution times. They are useful for comparing growth, but should not be treated as benchmark results.

Recommendations

- M1: redesign the pairwise duplicate scan. Use a hash set keyed by normalized (title, seller), or sort by the composite key and scan adjacent entries. Expected processing can be reduced to $O(n)$ with hashing or $O(n \log n)$ with sorting.
- M2: the doubling-window loop is $\Theta(n)$, but its count still exceeds the strict 10-minute budget at the stated rate. Process only necessary request buckets, use aggregated counters, and benchmark actual operation costs.
- S2: keep the sorted index and binary search for lookup; ensure index maintenance cost is considered separately.

- M4: remove replicated recursive partitions as proposed. The change improves $\Theta(n^2)$ to $\Theta(n \log n)$, but the given 10 ops/second model still predicts more than 10 minutes at one million records. Benchmark and consider parallelism, caching, or incremental computation.
- M5: investigate incremental analytics, pre-aggregation, and avoiding repeated sorting at every recursion level. Measure actual run time before production decisions.

10. Program Execution and Verification

Language: Java. The program uses only the standard library and can be compiled with JDK 21 or later.

```
javac ShopSphereScalabilityAudit.java
java ShopSphereScalabilityAudit
```

The attached output file is the captured console output from a successful local compilation and execution. The code prints formula checks and model-based time estimates. It does not claim that the estimates are measurements from a live ShopSphere system.

Source code screenshot:

Java Source Code — Page 1

```
import java.util.Locale;

/**
 * ShopSphere Scalability Audit
 * CSA06 - Design and Analysis of Algorithms, CO1 AT1
 *
 * Counts are theoretical estimates based on the models stated in the question.
 * The program does not access real e-commerce data.
 */
public class ShopSphereScalabilityAudit {
    private static final double OPS_PER_SECOND = 10.0;
    private static final double NIGHTLY_WINDOW_SECONDS = 10.0 * 60.0;

    private static double log2(double n) {
        return Math.log(n) / Math.log(2.0);
    }

    private static String format(double value) {
        return String.format(Locale.US, "%,.2f", value);
    }

    private static String duration(double operations) {
        double seconds = operations / OPS_PER_SECOND;
        if (seconds < 60) return format(seconds) + " seconds";
        if (seconds < 3600) return format(seconds / 60) + " minutes";
        if (seconds < 86400) return format(seconds / 3600) + " hours";
        if (seconds < 365.25 * 86400) return format(seconds / 86400) + " days";
        return format(seconds / (365.25 * 86400)) + " years";
    }

    private static void printEstimate(String module, double operations, String note) {
        System.out.printf("%-25s %18s ops | %18s | %s%n",
```

Java Source Code — Page 2

```
                module, format(operations), duration(operations), note);
    }

    public static void main(String[] args) {
        long nCurrent = 100_000L;
        long nFuture = 1_000_000L;
        long nM2Check = 1_024L;

        System.out.println("============================================================");
        System.out.println("       SHOPSPHERE E-COMMERCE SCALABILITY AUDIT");
        System.out.println(" CSA06 - Design and Analysis of Algorithms | CO1 AT1 | Java");
        System.out.println("============================================================");
        System.out.println("Assumption: 10 basic operations per second; nightly window = 600 seconds.");
        System.out.println();

        System.out.println("(b) ITERATIVE MODULES");
        double m1Current = (double) nCurrent * (nCurrent - 1) / 2.0;
        double m1Future = (double) nFuture * (nFuture - 1) / 2.0;
        System.out.println("M1 exact comparisons = n(n-1)/2");
        System.out.println("  n = " + nCurrent + ": " + format(m1Current));
        System.out.println("  n = " + nFuture + ": " + format(m1Future));
        System.out.println("  Time: Theta(n^2); auxiliary space: O(1), excluding stored duplicate marks.");
        System.out.println();

        int m2Iterations = (int) Math.ceil(log2(nM2Check));
        long m2CheckCount = (1L << m2Iterations) - 1L;
        int m2FutureIterations = (int) Math.ceil(log2(nFuture));
        long m2FutureCount = (1L << m2FutureIterations) - 1L;
        System.out.println("M2 exact Audit count = 2^ceil(log2(n)) - 1 (for integer n >= 1)");
        System.out.println("  n = " + nM2Check + ": " + m2CheckCount + " Audit operations");
        System.out.println("  n = " + nFuture + ": " + m2FutureCount + " Audit operations");
        System.out.println("  Team claim Theta(n log n): INCORRECT; tight bound is Theta(n).");
```

Program output screenshot:

Program Output — Page 1

```
============================================================
       SHOPSPHERE E-COMMERCE SCALABILITY AUDIT
 CSA06 - Design and Analysis of Algorithms | CO1 AT1 | Java
============================================================
Assumption: 10 basic operations per second; nightly window = 600 seconds.

(b) ITERATIVE MODULES
M1 exact comparisons = n(n-1)/2
  n = 100000: 4,999,950,000.00
  n = 1000000: 499,999,500,000.00
  Time: Theta(n^2); auxiliary space: O(1), excluding stored duplicate marks.

M2 exact Audit count = 2^ceil(log2(n)) - 1 (for integer n >= 1)
  n = 1024: 1023 Audit operations
  n = 1000000: 1048575 Audit operations
  Team claim Theta(n log n): INCORRECT; tight bound is Theta(n).

(c) RECURSIVE BINARY SEARCH
Recurrence: T(n) = T(n/2) + 1, T(1) = 1.
Backward substitution: T(n) = T(n/4) + 2
  = T(n/8) + 3 = ... = T(n/2^k) + k.
Set n/2^k = 1 => k = log2(n); T(n) = 1 + log2(n) for powers of 2.
```

Program Output — Page 2

```
  At n = 1000000, log2(n) = 19.93; practical comparison estimate is about 20.
  Time: Theta(log n); recursive auxiliary stack: Theta(log n).
  Iterative binary search auxiliary space: Theta(1).

(d) MASTER THEOREM RESULTS
M4 current: T(n)=4T(n/2)+n; a=4, b=2, f(n)=n, n^(log_b(a))=n^2.
  Master Theorem Case 1 => Theta(n^2).
M4 redesign: T(n)=2T(n/2)+n; a=2, b=2, f(n)=n, n^(log_b(a))=n.
  Master Theorem Case 2 => Theta(n log n).
M5: T(n)=2T(n/2)+n log n => extended Case 2 => Theta(n(log n)^2).

(f) REPORT GENERATOR BOUND
f(n)=3n^2+10n log2(n)+50 is Theta(n^2), and is not O(n).
For n >= 1, log2(n) <= n, so f(n) <= 3n^2+10n^2+50n^2 = 63n^2.
Also f(n) >= 3n^2. Thus c1=3, c2=63, n0=1 prove Theta(n^2).
f(n)/n >= 3n grows without bound, so f(n) is not O(n).

(g) FUTURE CATALOGUE: n = 1,000,000
Estimated dominant-operation counts and time at 10 operations/second:
M1 duplicate detector     499,999,500,000.00 ops |     1,584.40 years | EXCEEDS 10-minute window
M2 rate limiter                 1,048,575.00 ops |          1.21 days | EXCEEDS 10-minute window
S2 binary search                       20.00 ops |       2.00 seconds | WITHIN 10-minute window; about 20 comparisons
```

```
Program Output — Page 3

M4 current (n^2 proxy)    1,000,000,000,000.00 ops |     3,168.81 years | EXCEEDS 10-minute window; quadratic proxy
M4 redesign (n log n)          19,931,568.57 ops |       23.07 days | EXCEEDS 10-minute window at the stated 10 ops/s
M5 analytics (n log^2 n)      397,267,425.63 ops |        1.26 years | EXCEEDS 10-minute window; asymptotic proxy

Decision: redesign M1 using hashing/sorting; optimize M2 to process only
the required request window; retain indexed binary search for S2.
Remove replication in M4, then benchmark and optimize constants/parallelism.
For M5, consider more efficient sorting/aggregation and incremental reports.
Note: asymptotic proxies estimate growth; real timings require benchmarking.
============================================================
```

11. Conclusion

The audit shows that asymptotic growth has a major effect on the one-million-product catalogue. M1 is the most urgent redesign because its pairwise comparisons grow quadratically. M2 is linear, but its operation count still exceeds the given nightly window at the stated server rate. Binary search S2 is the most scalable lookup option among the two search versions. Removing replication improves M4 from quadratic to n log n growth, while M5 has n log-squared-n growth. All estimated durations should be validated with real benchmarks and measured operation costs.

12. Submission Checklist

- Replace the student name and registration number placeholders.
- Check that the Java file compiles and runs on your system.
- Review the report and be ready to explain the derivations during viva voce.
- Upload the CO1_AT1 folder to a GitHub repository with the README at repository root.
- Submit only the GitHub repository URL through Google Classroom, as instructed.